Projeto e Implementação de uma Ferramenta de Compilação para a Linguagem TPP

Ricardo de Paula Xavier
Departamento Acadêmico de Computação (DACOM)
Universidade Tecnológica Federal do Paraná (UTFPR)

Resumo

Este artigo apresenta o desenvolvimento de um analisador léxico para a linguagem C-. O projeto utiliza uma Máquina de Moore para implementar o analisador léxico, que é responsável por identificar tokens a partir de um código fonte. O trabalho também aborda as etapas de teste e validação do analisador, fornecendo uma visão geral do processo de análise léxica.

1 Introdução

A análise léxica é a primeira fase do processo de compilação, responsável por transformar o código-fonte em uma sequência de tokens, que são unidades significativas para a análise sintática. Este relatório descreve a implementação de um analisador léxico (analex) para uma linguagem de programação simples, utilizando autômatos de Moore para reconhecer padrões no código-fonte.

O analisador léxico foi implementado em Python, utilizando a biblioteca automata para definir o autômato de Moore. O código-fonte é lido, caracter por caracter, e o analisador léxico identifica tokens como palavras reservadas, operadores, números e identificadores, além de detectar erros léxicos.

2 Análise Léxica

A análise léxica é realizada por um autômato de Moore, que é um modelo de máquina de estados finitos onde cada estado possui uma saída associada. O autômato é definido com base nos seguintes componentes:

- **Estados:** Representam os diferentes estágios do processo de reconhecimento de tokens.
- **Alfabeto de entrada:** Conjunto de caracteres válidos na linguagem.
- **Tabela de transições:** Define como o autômato muda de estado com base no caractere de entrada.
- **Tabela de saída:** Associa cada estado a um token ou a uma ação específica.

O autômato implementado reconhece os seguintes tokens:

- **Palavras reservadas:** IF, ELSE, INT, RETURN, VOID, WHILE, FLOAT.
- **Operadores:** PLUS, MINUS, TIMES, DIVIDE, LESS, LESS_EQUAL, GREATER, GREATER_EQUAL, DIFFERENT, EQUALS, ATTRIBUTION.
- **Delimitadores:** SEMICOLON, COMMA, LPAREN, RPAREN, LBRACKETS, RBRACKETS, LBRACES, RBRACES.
- **Números:** NUMBER.

Além disso, o analisador léxico detecta caracteres inválidos e emite erros léxicos apropriados.

Corresponding author(s): Random Author, University of Somewhere, random.author@uni-somewhere.edu

2.1 Implementação do Autômato

O autômato de Moore é um modelo de máquina de estados finitos onde cada estado possui uma saída associada. No contexto da análise léxica, ele é utilizado para reconhecer padrões no código-fonte e gerar tokens correspondentes. A implementação do autômato de Moore no código analex.py é feita utilizando a biblioteca automata, que facilita a definição dos estados, transições e saídas.

2.2 Definição dos Estados

O autômato de Moore implementado possui 57 estados, nomeados de q0 a q56. Cada estado representa uma etapa no processo de reconhecimento de tokens. Por exemplo:

q0: Estado inicial, onde o autômato começa a processar o código-fonte.

q2: Estado final para o token IF.

q6: Estado final para o token ELSE.

q18: Estado final para o token VOID.

q28: Estado final para o token LESS.

q49: Estado final para o token NUMBER

Definidos como:

```
moore = Moore(
  ['q0', 'q1', 'q2', 'q3', 'q4', 'q5', 'q6', 'q7', 'q8', 'q9', 'q10',
   'q11', 'q12', 'q13', 'q14', 'q15', 'q16', 'q18', 'q19', 'q20', 'q21',
   'q22', 'q23', 'q24', 'q25', 'q26', 'q27', 'q28', 'q29', 'q30', 'q31',
   'q32', 'q33', 'q34', 'q35', 'q36', 'q37', 'q38', 'q39', 'q40', 'q41',
   'q42', 'q43', 'q44', 'q45', 'q46', 'q47', 'q48', 'q49', 'q50', 'q51',
   'q52', 'q53', 'q54', 'q55', 'q56'],
  ...
)
```

2.3 Alfabeto de Entrada

O alfabeto de entrada é o conjunto de caracteres válidos que o autômato pode processar. No código, ele é definido como:

```
['i', 'e', 'r', 'v', 'w', 'f', 'n', 'l', 't', 's', 'o', 'a', 'u', 'd', 'h',
 '+', '-', '*', '/', '<', '=', '>', '!', ';', ',', '(', ')', '[',
 ']', '{', '}', ' ', '\n', '0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
```

Esses caracteres incluem letras, dígitos, operadores e delimitadores. O autômato ignora caracteres inválidos e emite um erro léxico quando eles são encontrados.

2.4 Tabela de Transições

A tabela de transições define como o autômato muda de estado com base no caractere de entrada. Por exemplo:

No estado q0, se o caractere for i, o autômato vai para o estado q1.

No estado q1, se o caractere for f, o autômato vai para o estado q2, que é o estado final para o token IF.

A tabela de transições é definida no código da seguinte forma:

```
{
  'q0': { 'i': 'q1', 'e': 'q3', 'r': 'q10', 'v': 'q15', 'w': 'q19', '+':  'q23',
          'a': 'q50', 'u': 'q50', 'd': 'q50', 'h': 'q50', 's': 'q50', 'o': 'q50'
              ,
          't': 'q50', 'l': 'q50', 'n': 'q50', 'f': 'q51', '-': 'q24', '*':'q25',
          '/': 'q26', '<': 'q27', '>': 'q30', '=': 'q33', '!': 'q34', ';':'q44',
          ',': 'q45', '(': 'q37', ')': 'q38', '[': 'q39', ']': 'q40', '{':'q41',
          '}': 'q42', '0': 'q48', '1': 'q48', '2': 'q48', '3': 'q48', '4':'q48',
          '5': 'q48', '6': 'q48', '7': 'q48', '8': 'q48', '9': 'q48', ' ': 'q0',
          '\n': 'q0' },
  'q1': { 'f': 'q2', 'n': 'q7' },
  'q2': { '' : 'q0' },
  ...
}
```

2.5 Tabela de Saída

A tabela de saída associa cada estado a um token ou a uma ação específica. Por exemplo:

O estado q2 está associado ao token IF.

O estado q6 está associado ao token ELSE.

O estado q28 está associado ao token LESS.

A tabela de saída é definida no código da seguinte forma:

```
{
  'q0': '', 'q1': '', 'q2': 'IF', 'q3': '', 'q4': '', 'q5': '', 'q6': 'ELSE',
  'q7': '', 'q8': '', 'q9': 'INT', 'q10': '', 'q11': '', 'q12': '', 'q13': '',
  'q14': 'RETURN', 'q15': '', 'q16': '', 'q17': '', 'q18': 'VOID', 'q19': '',
  'q20': '', 'q21': '', 'q22': 'WHILE', 'q23': 'PLUS', 'q24': '', 'q25': 'TIMES'
      ,
  'q26': 'DIVIDE', 'q27': '', 'q28': 'LESS', 'q29': 'LESS_EQUAL', 'q30': '',
  'q31': 'GREATER', 'q32': 'GREATER_EQUAL', 'q33': '', 'q34': '', 'q35': 'EQUALS
      ',
  'q36': 'DIFFERENT', 'q37': 'LPAREN', 'q38': 'RPAREN', 'q39': 'LBRACKETS',
  'q40': 'RBRACKETS', 'q41': 'LBRACES', 'q42': 'RBRACES', 'q43': 'ATTRIBUTION',
  'q44': 'SEMICOLON', 'q45': 'COMMA', 'q46': '', 'q47': '', 'q48': '',
  'q49': 'NUMBER', 'q50': '', 'q51': '', 'q52': '', 'q53': '', 'q54': '',
  'q55': 'FLOAT', 'q56': 'MINUS',
}
```

2.6 Funcionamento do Autômato

O autômato começa no estado inicial q0. À medida que os caracteres do código-fonte são lidos, o autômato muda de estado com base na tabela de transições. Quando um estado final é alcançado, o token correspondente é emitido.

Por exemplo, ao processar a palavra if:

O autômato começa em q0.

Ao ler i, ele vai para q1.

Ao ler f, ele vai para q2.

O estado q2 é um estado final associado ao token IF, então o token IF é emitido.

2.7 Tratamento de Erros

O autômato também detecta caracteres inválidos. Se um caractere não estiver no alfabeto de entrada, o analisador léxico emite um erro léxico. Isso é feito na função caractere_invalido:

```
1 def caractere_invalido(caractere):
2     return caractere not in caracteres_validos
```

Se um caractere inválido for encontrado, o analisador léxico adiciona um token de erro à lista de tokens:

```
1 if caractere_invalido(simbolo):
2     tokens.append(('ERROR', 'ERR-LEX-INV-CHAR'))
3     continue
```

2.8 JFLAP da Máquina de Moore

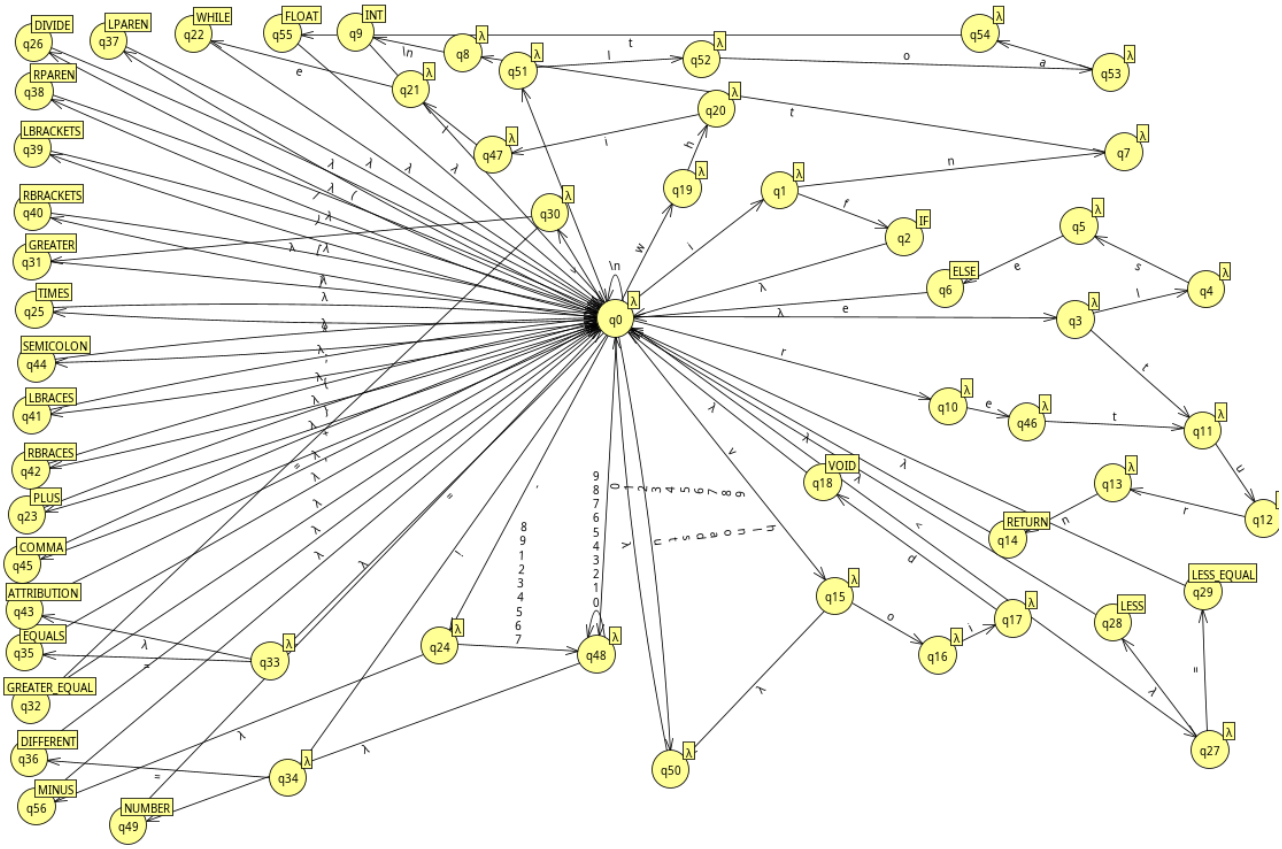


Figura 1: Figura do JFLAP da Maquina de Moore

3 Análise Sintática

A análise sintática é a segunda fase do processo de compilação, onde a sequência de tokens é verificada em relação à gramática da linguagem.

4 Análise Semântica

A análise semântica é a terceira fase ela verifica se a estrutura do programa faz sentido em termos de tipos e operações. Por exemplo, verifica se variáveis foram declaradas antes do uso e se operações são realizadas com tipos compatíveis.

5 Geração de Código

Para garantir o correto funcionamento do Analisador Léxico, foram desenvolvidos testes automatizados utilizando a biblioteca pytest. Esses testes estão implementados no arquivo analex_test.py e verificam se a saída gerada pelo analisador corresponde à saída esperada, armazenada em arquivos com a extensão .lex.out.

5.1 Execução dos Testes

- **Busca dos Arquivos de Teste:** O script busca os arquivos de teste no diretório especificado.
- **Execução do Analisador Léxico:** Para cada arquivo de entrada, o Analisador Léxico é executado, processando o código-fonte e gerando uma lista de tokens.
- **Comparação com a Saída Esperada:** A saída gerada é comparada com a saída esperada armazenada nos arquivos `.lex.out`.

5.2 Comando utilizado para executar um teste específico:

```
python analex.py tests/prog-007.cm -k
```

A saída irá mostrar os TOKENS do programa.

- **Comando para rodas todos os testes:**

```
pytest ou pytest -v
```

A saída mostrará qual dos testes passaram pelo analex:

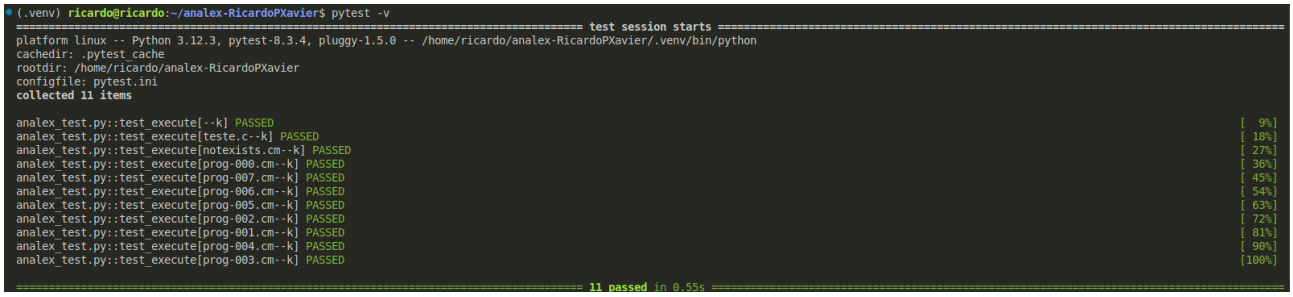


Figura 2: Saída com pytest -v

5.3 Correções Durante os Testes:

Durante a fase de testes, foram identificadas inconsistências nos arquivos de saída esperada (especificamente nos Testes 4, 5, 6 e 7). Esses erros impediam a validação correta do Analisador Léxico. Para resolver o problema, foram feitas correções nos arquivos de saída, garantindo que eles refletissem o comportamento esperado do código. Essas correções permitiram a execução bem-sucedida dos testes e a validação completa do Analisador Léxico.

Referências

- Louden (2004) aborda os princípios e práticas de compiladores.
- Menezes (2011) explora linguagens formais e autômatos.
- Rodger (2006) apresenta o JFLAP, uma ferramenta interativa para autômatos.
- Sipser (2007) trata da teoria da computação.
- GitHub Classroom Assignment